Lab 2: Threat Modelling in Practice

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

In Lab 1 you enumerated the attack surface of the course VM: what listens, what is privileged, what is scheduled, what is exposed at rest. In this lab you turn that flat inventory into a **threat model**: a structured argument about *who* would attack the machine, *what* is worth protecting, *how* the attack proceeds, and *which weaknesses matter most*. You will name an attacker and value the assets, draw a data-flow diagram, apply STRIDE element by element, build an attack tree for one concrete goal, and end with a prioritised list. The deliverable is the model itself: a document you could hand to whoever operates the machine.

This lab is analysis, done on paper or in a diagram tool: you build a model, you do not exploit anything. You may run read-only commands to inspect the machine's configuration, but none of them attacks it. Bring your Lab 1 notes; you cannot do this lab without them.

Contents

- Learning Objectives
- Background: three instruments
- Before the lab (at home, ~20 min)
- Part A: Attacker and assets
- Part B: Data-flow diagram of the machine
- Part C: STRIDE, element by element
- Part D: An attack tree
- Part E: Prioritise, and measure
- Reflection
- References
- Deliverables: Lab report 1 (individual)

Learning Objectives

By completing this lab, students will be able to:

1. Characterise an attacker (position, goal, motivation) and rank a system's assets by value.
2. Represent a system as a data-flow diagram with explicit trust boundaries.
3. Apply the STRIDE taxonomy element by element to derive threats systematically rather than by inspiration.
4. Build an attack tree that decomposes an attacker goal into concrete steps, and read off the cheapest **attack set** (the minimal sub-tree: cheapest child at OR nodes, all children summed at AND nodes).
5. Turn an unordered set of weaknesses into a *prioritised* model, and justify the ordering.
6. Derive a one-line security **policy** statement from a threat model, and tell it apart from the mechanism that enforces it.

These map to FUC outcomes LO2 (defining protection policies from assets, actors, motivations and threats) and LO4 (recognising a system's security posture).

Background: three instruments

Data-flow diagram (DFD). A DFD shows four things: **external entities** (actors outside your control), **processes** (running code), **data stores** (data at rest), and **data flows** (arrows between them). Over the top you draw **trust boundaries**: dashed lines crossed whenever data moves between things at different privilege or trust levels. Threats concentrate on the boundaries.

The notation is small, and the whole of it is this:

Element	Drawn as
External entity	a rectangle
Process	a circle or an ellipse
Data store	two parallel horizontal lines, or an open-ended rectangle
Data flow	an arrow, labelled with what it carries
Trust boundary	a dashed line drawn across the flows it cuts

STRIDE. A checklist of six threat categories, applied to each element of the DFD: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service, **E**levation of privilege. Its value is **systematic coverage**, not a guarantee of completeness: you ask all six of every element instead of only imagining the attacks you already know, which reduces predictable omissions but does not prove the model complete (threats from *composition* can still be missed).

Not every category applies to every kind of element, and knowing which is what turns ”ask all six” from a chore into a short list. An x means the category applies:

Element type	S	T	R	I	D	E
External entity	x		x			
Process	x	x	x	x	x	x
Data store		x	x	x	x	
Data flow		x		x	x	

A **data flow** can be altered in transit (T), read in transit (I) or cut (D). It cannot be spoofed, because it has no identity of its own; it cannot repudiate, because it takes no action; and it cannot have its privilege raised, because it holds none.

⚠ **The table says where to look, not what you will find.** A cell marked x still needs a concrete threat or a reasoned ”n/a”; a blank cell is one fewer question, not a guarantee.

Attack tree. A goal at the root, decomposed into sub-goals and finally into concrete actions at the leaves, with AND/OR nodes. Leaves are always **actions the attacker takes**, never defences. Annotate leaves with cost or difficulty; the attacker's likely route is the **cheapest attack *set*** -- at an OR node take the cheapest child, at an AND node you need *all* children so sum them. That set is a single path only when the tree has no AND nodes; otherwise it is a sub-tree.

Lecture 2 works this tree for `root` on the **db–backup host**, a different machine from yours. Read it for the shape, not for the answer: the AND node is the one that makes ”cheapest” interesting, because both of its children have to be paid for.

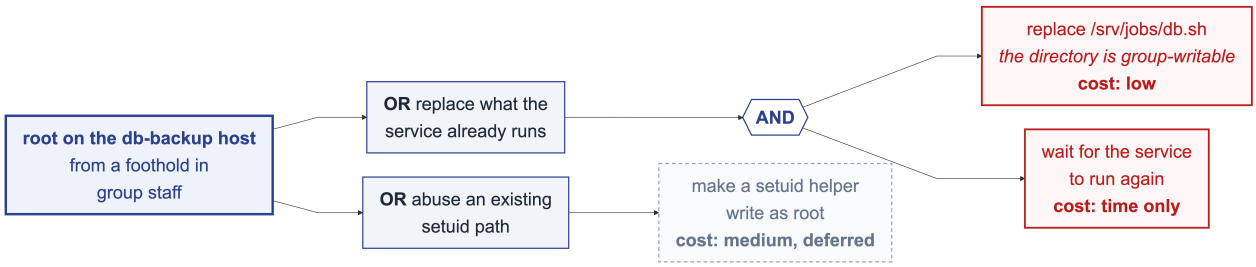


Figure 1: An attack tree for root on the db-backup host, as worked in Lecture 2

Before the lab (at home, ~20 min)

This lab is analysis, not setup, but come prepared:

- bring your **Lab 1 attack-surface inventory** (you cannot do this lab without it);
- re-read Lecture 2 if you have already had it (data-flow diagrams, STRIDE, attack trees). **If your lecture comes after this session, you lose nothing**: the Background section below defines all three instruments, and this lab does not depend on anything else from the lecture.

Preparation questions (submit with Lab report 1, this lab's report):

- **P1.** Name the four elements of a data-flow diagram.
- **P2.** In one sentence, what does STRIDE give you that an ad-hoc list of ”problems” does not?

Part A: Attacker and assets

Before modelling *how* the machine is attacked, state *who* attacks it and *what* is worth protecting. A threat model with an unnamed attacker and unvalued assets is untestable.

Exercise 1: Attacker profile. A threat model needs a specific attacker, because a system is only secure *against someone*. The owner of this VM, the `student` account, is a full administrator. That is expected and **out of scope**: we do not model the legitimate owner attacking their own machine. Model instead an attacker who already has a **foothold**: the ability to run code on the machine as a low-privilege account that is not the owner. Here, the foothold is the `svc-report` service account, the identity behind the port-8081 service you found in Lab 1.

Model `svc-report` specifically, not ”any low-privilege user”. This account runs the service that reads `report.conf`, and it has its own `sudo` rights. The model depends on both.

Find out what `svc-report` can do with `sudo`. A service account is rarely given nothing, and what it is given is easy to overlook. Check what `sudo` lets `svc-report` run on your machine.

Hint: the `sudo` manual (`man sudo`) describes how to list the privileges of another user. Run it from your `student` account.

Record exactly what your machine reports, not what you expect it to say. Do **not** try to exploit what you find. Write it into the model as a permitted capability **whose reach you have not measured**, and note the open question, as you did with `backup-tool` in Lab 1: *what can this permitted command actually reach?*

Then write down this attacker's:

- **position:** where does the attacker start? What can they already do on the machine (which account, which privileges), and how can they reach it from the network?
- **goal(s):** what do they want to achieve?
- **motivation:** why would someone target a machine like this? One or two sentences.

Exercise 2: Assets. List the assets of this machine worth protecting; you will find candidates in your Lab 1 observations. Rank them by value **to the attacker you defined in Exercise 1**, with one sentence of justification each. This ranking feeds your prioritisation in Part E.

Part B: Data-flow diagram of the machine

Exercise 3. Draw a DFD of the course VM as you found it in Lab 1. Start from these elements, which come from your Lab 1 inventory:

- external entities: a **remote client** on the network, and the **local user** `student`;
- processes: the **port-8081 service** (`report.service`, running as `svc-report`), the **root cron job**, and the **shell** an attacker drives from their foothold;

- data stores: the **credentials file** (`/etc/report/report.conf`), the **maintenance script** (`/opt/maint/rotate.sh`), the **setuid binary** (`/usr/local/bin/backup-tool`), and the **sudo policy** (`/etc/sudoers.d/`).

Then add the parts that need your judgement:

- **Data flows.** Connect the elements. Draw only flows you can support with something you observed in Lab 1, and label each one with what it carries. Keep processes, flows and data stores as *separate* elements: `root cron job` is a process, an arrow from it to a file is a flow.
- **Trust boundaries.** Draw the dashed lines where trust or privilege changes. Mark at least three, and name each one. One of them is the VM's network edge; the other two are yours to find.

The remote client is an assumption, not an observation. In Lab 1 you saw the service *listening* on `0.0.0.0:8081`, but you only reached it from the VM itself (`127.0.0.1`). You did not show that other machines can reach it: that depends on interfaces, routing, the firewall and the hypervisor's network mode. Mark the remote client as an **assumption** in your diagram. Modelling a plausible remote attacker is legitimate; presenting reachability as a measured fact is not.

3a. Submit the diagram. In two or three sentences, identify which boundary you expect to be the *most* attacked, and why.

Part C: STRIDE, element by element

Exercise 4. Build a STRIDE table. The rows below are a **starter** covering the seeded focus items; **extend the table with the remaining DFD elements from Part B** (at least the remote client and the attacker's shell) so that every element of your diagram appears. The word in brackets is the DFD element type (external entity, process, flow or data store). STRIDE is applied to each element separately, so a process and a flow that share a name are two rows. ⚠ **The flow row below is a placeholder showing the shape of a flow row: replace it with the flows you actually drew in Part B, one row each.** For each cell either write a concrete threat for that (element, category) pair, or ”n/a” with a word on why none applies.

Element	S	T	R	I	D	E
report.service (:8081) (process)						
root cron job (process)						
(add: a flow you drew, named `source → destination`, with what it carries) (flow)						
backup-tool, the setuid binary (data store)						
report.conf (data store)						
rotate.sh (data store)						
sudo policy: `svc-report` grant (data store)						
(add: remote client) (external entity)						
(add: attacker shell / foothold) (process)						

You do not need to know how each threat is exploited: several mechanisms arrive in later blocks. You do need to name each threat under the correct STRIDE category.

4a. Pick the three cells you judge most serious. For each, name the STRIDE category, the element, and one sentence on the harm if realised.

4b. STRIDE is a **completeness aid, not a guarantee** (it reduces predictable omissions; it does not prove the model complete, and threats that emerge from *composition* can still be missed). Name one threat you found *only because you worked through the grid systematically*: one you would probably have missed had you simply listed ”problems” as in Lab 1.

Part D: An attack tree

Exercise 5. Choose **one** of the two goals below. Both start from the unprivileged foothold of Part A: an attacker running code as `svc-report`, not as the `student` owner.

G1: obtain `root` on the VM.

Before you start: the `root` account on this image is **locked**. In `/etc/shadow` its password field is `*`, which is not a password hash and matches no possible password, so nobody can log in as root with a password. For G1 this means that *reading* `/etc/shadow` gives the attacker nothing to crack: it is Information Disclosure, **not** a route to root. A real G1 leaf has to *run code* as root or *write* a file root will act on.

G2: read the contents of `report.conf`.

Answer this before you choose G2. Your attacker runs as `svc-report`, and `svc-report` is the identity the service uses to read `report.conf` on every request. Does taking away the file's world-readable bit stop them?

If your answer is that the goal is already met from the starting position, do not draw a one-leaf tree. Say that in one line, then re-state G2 as the goal that is *not* already met and build the tree for that one. What the attacker would still have to defeat is the exercise.

Whatever you conclude, describe a local user for whom the answer would be different, and say what makes them different.

Build an attack tree for your chosen goal. Root = the goal. Decompose it into sub-goals with AND/OR nodes, down to leaves that are concrete **attacker actions** against *this* machine, referencing items from your DFD. Annotate each leaf with a rough difficulty (low / medium / high), and mark it ”deferred” if it needs a mechanism you have not yet been taught. ⚠ **5a asks you to *sum* these at an AND node, and low/medium/high do not add up on their own. State the convention you use** (a number per level, or a rule such as ”two lows make a medium”): any defensible convention is accepted, an unstated one is not.

5a. Read off the cheapest attack set: at an **OR** node take the cheapest child, at an **AND** node sum all the children (see Background). This is not simply ”the shortest path”. State your answer as the ordered steps of that minimal set. If a step depends on something you have not measured, keep it as a ”deferred” leaf instead of assuming the answer.

5b. A G1 tree probably contains at least one ”deferred” leaf, a step whose mechanism is taught later. **If yours has one**, does the *shape* of the cheapest path depend on it, or does a path exist entirely from things you already understand? **If yours has none**, say so and say why not. Either way: what does the answer reveal about how much of this machine's risk is already legible to you after two labs?

Part E: Prioritise, and measure

Exercise 6. Produce a **ranked** list of the machine's weaknesses, most to least urgent to fix, based on Parts C and D. For each item give: a one-line description, the STRIDE category, the single change that would remediate it (name it; you need not know how to apply it yet), and a one-sentence justification of its rank.

6a. Defend your top-ranked item against the second: why does the first deserve attention before the second?

6b. Lab 1 claimed attack surface can be treated as a *measurable quantity*. Propose **one metric** for this machine's attack surface (for example: the number of network-reachable services running as root). It must be something you can *count* on the VM and *recount* after a change, defined precisely enough that someone else would get the same number. Give its current value for the VM, and the value it would have if your top-ranked remediation were applied.

6c. The lecture asks you to write the **policy line first**, or the mechanism quietly becomes the policy. You have just named a mechanism for your top-ranked item, so recover the policy now.

Read this first. A policy line says who may do what, not which flag to set. "`rotate.sh` should not be world-writable" is your mechanism restated; "no unprivileged user may modify anything root executes" is the policy it serves, and it also forbids things you have not thought of yet.

1. Write, in **one line**, the policy your remediation assumes: what is allowed on this machine, and by whom. Use the form of the example above.
2. Name **one other mechanism** that would enforce the same line (a different permission, a different owner, removing the program altogether).
3. In one sentence each, say what each mechanism allows that the other does not. If the two differ, the line was doing work the mechanism alone was not.

Reflection

Your reflection goes into Lab report 1, as an appendix, beside the reflection from Lab 1, the other lab in this block. It is read under C3; see `rubrics.md`.

In a short piece (200 to 300 words):

1. A threat model built *after* an attack tends to justify what the attacker already did. This lab asked you to build the model *before* exploiting anything. What does modelling-first buy a defender that modelling-after does not?
2. STRIDE, DFDs and attack trees are three different lenses on the same machine. Where did they *disagree* or surface different things, and does that make the set redundant, or complementary?

References

- A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014 (STRIDE and DFDs).
- Microsoft, "The STRIDE Threat Model", https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
- B. Schneier, "Attack Trees", *Dr. Dobb's Journal*, 1999.
- W. Du, *Computer Security: A Hands-on Approach*, 3rd ed., 2022: Ch. 1.

Deliverables: Lab report 1 (individual)

Lab report 1 is your answer to every part of this handout, submitted individually even though the bench work is done in your group of three. It carries:

- Parts A to E of this lab: the attacker profile and ranked assets, the DFD with its trust boundaries, the STRIDE table with your three most-serious cells, the attack tree and its cheapest attack set, and the ranked remediation list with your defence of the top item, the surface metric and the policy line;
- the preparation questions **P1 and P2 of both labs in this block**, Lab 1's and this one's. Lab 1's P1 is the screen capture with the date, as an image and not as pasted text;
- as an appendix, your **Lab 1 attack-surface inventory**;
- as a further appendix, the **reflections of both labs** (see Reflection), read under C3.

The diagram and the attack tree go inside the report, as images. A photograph of a hand drawing is fine if it is legible at A4; a file only a particular tool can open is not a submission.

Due on Blackboard by **Sunday 11 October**, one week after Block 1 ends; the exact time is the one on the Blackboard assignment, and the late-report rule in the rubrics applies.